



People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Houari Boumediene University of Science and Technology



Faculty of Electrical Engineering
Department of Power Electrical Engineering

Field: Science and Technology

Major : Power Electrical Engineering

Project report

Title:

Control Study of a Differential Drive Mobile Platform

PRESENTED BY :

AROUS Imad
KAIS Abdelmalek

Supervisor :

Pr. KHELOUI Abdelaziz

Academic Year: 2024/2025

LIST OF FIGURES

Figure 1: Kinematic diagram of the differential drive robot with two driving wheels.....	4
Figure 2: wheel-ground contact model without slipping.....	5
Figure 3: side view of caster wheel and no slip conditions.....	5
Figure I.1: Electromechanical model of a DC motor	7
Figure I.2: Functional block diagram of Motor in LaPlace Domaine	9
Figure I.3: Speed Control Diagram with a PID Controller	9
Figure I.4: Oscillations obtained at the stability limit	10
Figure I.5: System response after PID (Ziegler-Nichols method)	10
Figure II.2: Full vehical model	12
Figure II.3: Illustration of e_1 and e_2 Errors in a Robot Trajectory following control strategy ...	13
Figure II.4: Converting trajectory into speed input.....	14
FigureIII.1: Vehicle Model (Subsystem)	16
Figure III.2: trajectory – circular motion ($\omega_r \neq \omega_l$)	17
Figure III.3: XY trajectory – straight motion ($\omega_r = \omega_l$)	17
Figure III.4: Complete Control Model	17
Figure III.5: Trajectory Tracking:Robot vs Reference Path.....	18
Figure III.6: Wheel Speeds vs Time	18
Figure III.7: Linear Velocity $v(t)$	19
Figure III.8: Angular Velocity $\omega(t)$	19

TABLE OF CONTENTS

Introduction.....	4
Problem Statement	4
Specifications	5
General Assumptions	5
Section I: Electromechanical Modeling of the Mobile Platform	6
I.1 Introduction	7
I.2 Mechanical Equations of the Motors	7
I.3 Electrical Equations of the Motors	7
I.4 Laplace Transformation	7
I.5 Meaning of Symbols and Parameter Values Used.....	8
I.6 DC Motor Model	8
I.7 Speed Regulation.....	9
I.8 PID Tuning Using the Ziegler-Nichols Method	10
Section II: Control Strategy	11
II.1 Linear Velocity of the Robot:	12
II.2 Angular Velocity of the Robot:	12
II.3 Motion Model in the Plane:	12
II.4 Simple Command Strategy:	13
II.5 Advanced Control Strategy: Following a Given Trajectory:	13
II.6 Control and error computation equations	14
Section III: Simulation and results	15
III.1 Simulink Model:	16
III.2 Reference Trajectory Tracking :	18
III.3 Wheel Angular Speeds Evolution:	19
III.4 Linear Velocity Evolution	19
III.5 Angular Velocity Evolution:	20
III.6 Conclusion of the Simulation Results:	20
Conclusion	21
General Conclusion	22

Introduction:

In the field of mobile robots, platforms with independently driven wheels play a crucial role in the development of flexible, maneuverable, and efficient locomotion systems. Among the most common architectures are vehicles with two independently driven wheels (left and right), which enable differential control of the trajectory without requiring mechanical steering.

Traditionally, in conventional automobiles, the mechanical differential is an essential device that allows the driving wheels to rotate at different speeds particularly when cornering in order to ensure good traction and balanced tire wear. In modern robotic systems, this function can be digitally replicated by independently controlling the speed of each motor, thus enabling more flexible and precise control.

This type of robot, known as a differential-drive robot, enables movement in a two-dimensional plane through variation in the angular velocities of the left (w_l) and right (w_r) wheels. Through kinematic and dynamic modeling, it becomes possible to simulate the robot's behavior based on the applied motor torques, electrical voltages, induced currents, and the resistance of mechanical components.

From this perspective, the development of a simulation using MATLAB/Simulink becomes a valuable tool. It enables the study of how mechanical and electrical parameters affect the robot's trajectory, while also allowing for digital control capable of replacing the mechanical differential.

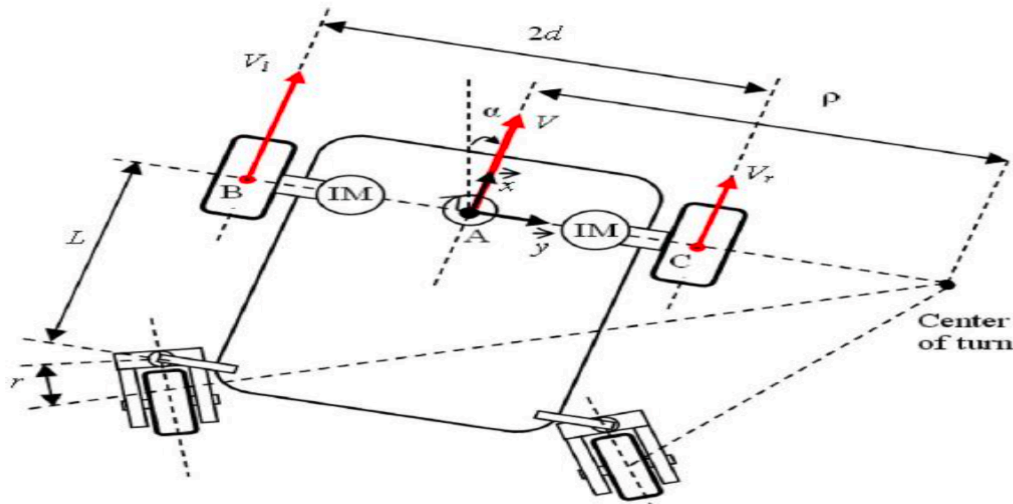


Figure 1: Kinematic diagram of the differential drive robot with two driving wheels

Problem Statement:

The objective of this project is to develop a simulation in MATLAB/Simulink of a mobile platform driven by two independent front electric motors. The main challenge is to control these two motors in such a way as to replace the mechanical speed differential, ensuring that the robot can navigate while maintaining both stability and precision in its movement.

Specifications :

1. Kinematic modeling of a mobile platform with two front drive wheels and two passive rear caster wheels.
2. Modeling and control of the wheel drives based on DC motors (direct current machines).
3. Development of a driving and control strategy for the platform using an electrical differential to manage wheel speed differences.
4. Numerical simulation study using Matlab/Simulink.

General Assumptions:

Before proceeding with the system modeling, it is important to formulate the simplifying assumptions and parameters used in this study. These assumptions help to narrow the modeling scope while preserving the essential characteristics of the mobile robot's dynamic behavior.

- The platform is driven by two front wheels: a left and a right wheel, each powered by a DC motor (direct current motor).
- The robot is also equipped with two passive rear caster wheels. These wheels are not motorized; they are free to rotate and adapt to the trajectory imposed by the drive wheels.
- The movement occurs on a horizontal plane **without slipping**:

-The wheels do not slide laterally, meaning there is no angular drift ($\alpha = 0$).

-The contact point P between the wheel and the ground is stationary relative to the wheel, i.e., the sliding velocity at the contact point is zero.

-No lateral force is generated by the rear caster wheels.

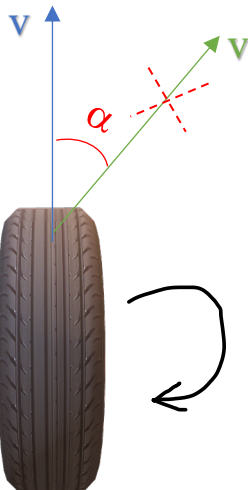


Figure 3: side view of caster wheel and no slip conditions

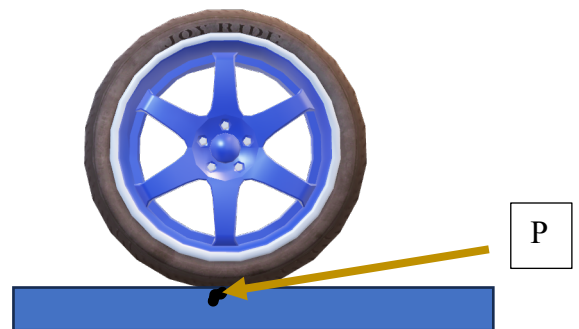


Figure 2: wheel-ground contact model without slipping

Section I:

Electromechanical

Modeling of the Mobile

Platform

I.1 Introduction:

This part of the work focuses on the electromechanical modeling of the mobile platform's locomotion system, which is driven by two independently controlled DC motors mounted on the left and right drive wheels. The goal is to establish differential equations that describe both the electrical and mechanical dynamics of the motors.

These equations are then transformed into the Laplace domain and represented through block diagrams, offering a structured basis for system analysis, control design, and simulation. This modeling step is essential for understanding the platform's dynamic behavior and forms the foundation for the control strategies and simulations developed in the subsequent stages of the project.

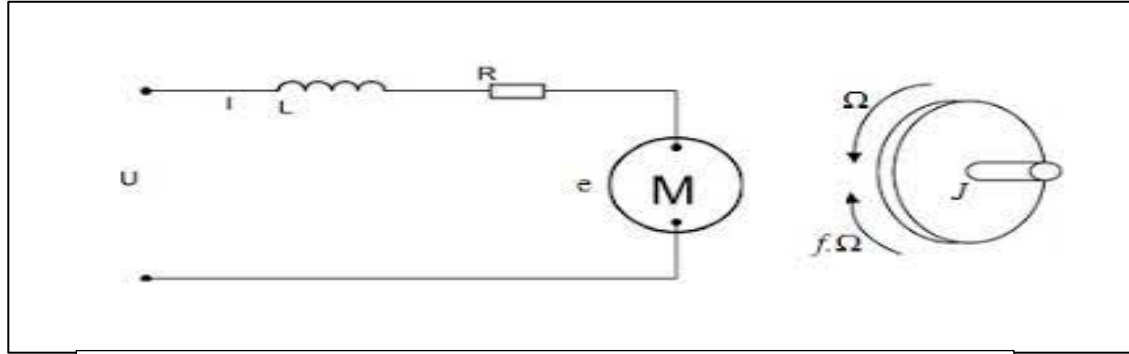


Figure I.1: Electromechanical model of a DC motor

I.2 Mechanical Equations of the Motors:

$$J \frac{d\omega(t)}{dt} + f\omega(t) = C_m(t) - C_r(t)$$

with :

$$C_m(t) = K_c i(t)$$

I.3 Electrical Equations of the Motors:

$$U(t) = Ri(t) + L \frac{di(t)}{dt} + K_e \omega(t)$$

I.4 Laplace Transformation:

After applying the Laplace transformation, the equations become:

$$C_m(p) - C_r(p) = (Jp + f)\omega(p)$$

$$U(p) = (R + Lp)i(p) + K_e \omega(p)$$

I.5 Meaning of Symbols and Parameter Values Used:

U(t) : Applied voltage to the motor (V)	$R = 84 \times 10^{-3} \Omega$
e(t) : Back electromotive force (V)	$L = 2,6H$
i(t) : Armature current (A)	$K_e = 1,1(V \cdot rad^{-1} \cdot s)$
$\omega(t)$: Angular velocity of the motor shaft (rad . s ⁻¹)	$Cr=0,1(N \cdot m)$
Cm(t) : Motor torque (N. m)	$K_c = 1,1(N \cdot m \cdot A^{-1})$
Cr(t) : Load Torque (N. m)	$f = 2,5 \times 10^{-6}(N \cdot m \cdot rad^{-1} \cdot s)$
f : Friction coefficient (N. m/rad.s ⁻¹)	$j = 10^{-4}(Kg \cdot m^2)$
J : Moment of inertia of the rotating parts (Kg. m ²)	$r = 0,1m$
L : Armature Inductance (H)	$L = 0.5m$
R : Armature Resistance (Ω)	
Ke : Back Electromotive Force Constant (V . rad ⁻¹ . s)	
Kc : Torque Constant (N. m. A ⁻¹)	

I.6 DC Motor Model:

Inputs : U(P),C_r(P)

Output : w(P)

Equation for the input voltage (U) and back electromotive force (E)

$$U(P) - E(P) = (R + L \times P) \times I(P)$$

$$U(P) - E(P) \times \frac{1}{R + L \times P} = I(P)$$

Relating current and torque:

$$C_m(P) = K_c \times I(P)$$

Equation for the mechanical dynamics of the motor:

$$C_m(P) - C_r(P) = (f + j \times P) \times w(P)$$

$$C_m(P) - C_r(P) \times \frac{1}{f + j \times P} = w(P)$$

Back EMF relation :

$$E(P) = K_e \times w(P)$$

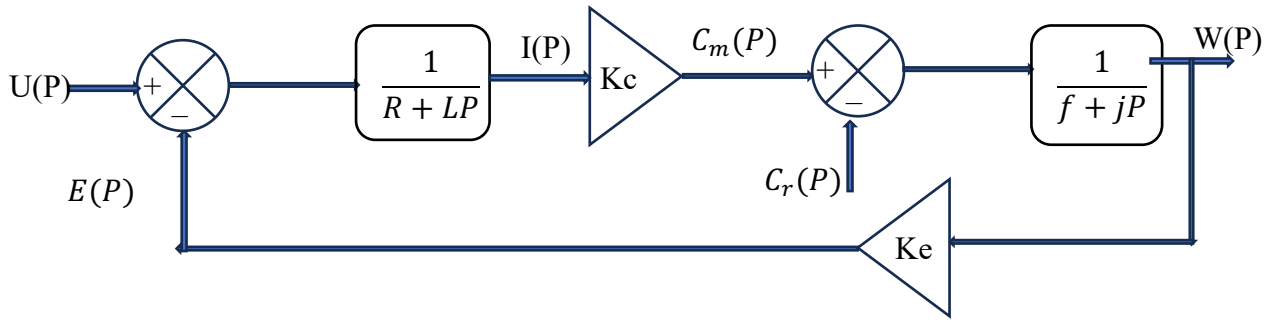


Figure I.2: Functional block diagram of Motor in LaPlace Domaine

I.7 Speed Regulation:

To ensure precise control of the rotational speed of the DC motors driving the mobile platform, a **PID controller (Proportional – Integral – Derivative)** is used. This controller minimizes the error between the desired speed (setpoint) and the actual speed by adjusting the input voltage applied to the motor.

The mathematical expression of the PID controller in the **time domain** is given by:

$$V(t) = K_p \times e(t) + K_i \times \int e(t)dt + K_d \times \frac{de(t)}{dt}$$

The corresponding expression in the Laplace domain is

$$V(P) = K_p \times e(P) + \frac{K_i}{P} \times e(P) + P \times K_d \times e(P)$$

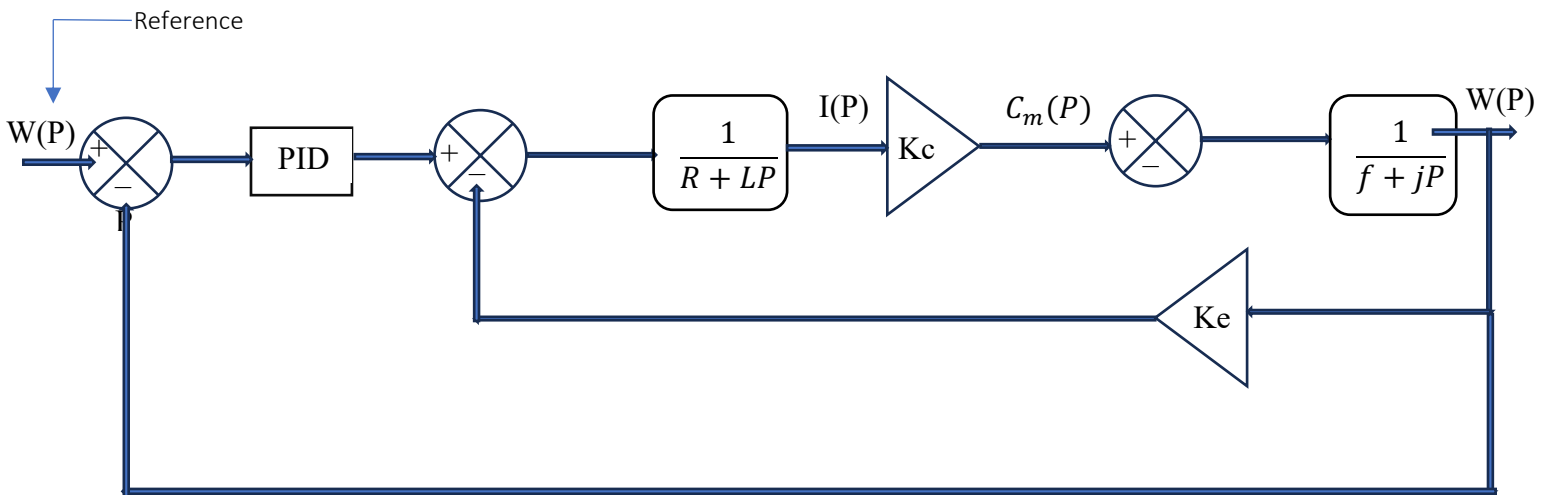


Figure I.3: Speed Control Diagram with a PID Controller

I.8 PID Tuning Using the Ziegler-Nichols Method:

Principle of the Method:

The Ziegler-Nichols method involves gradually increasing the proportional gain K_p until the system output begins to exhibit sustained and stable oscillations. The gain at this point is called **Koscillation** (Goscillation), and the period of these oscillations is noted as T .

These two values are then used to calculate the PID controller parameters using standard tuning formulas.

$$K_p = \frac{G}{1.7}$$

$$K_i = \frac{0.85 \times T}{G}$$

$$K_d = \frac{T \times G}{13.3}$$

Thus, we obtain :

$$k_p = \frac{G}{1.7} = 132.35$$

$$K_i = \frac{0.85 \times T}{G} = 1.64 \times 10^{-3}$$

$$K_d = \frac{T \times G}{13.3} = 7.359$$

Experimental example :

$$\text{Goscillation} = 225$$

$$T = 0.435\text{s}$$

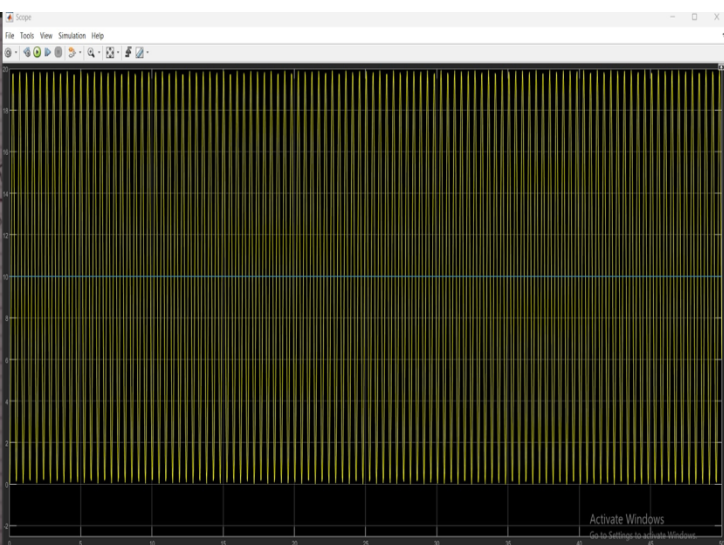


Figure I.4: Oscillations obtained at the stability limit

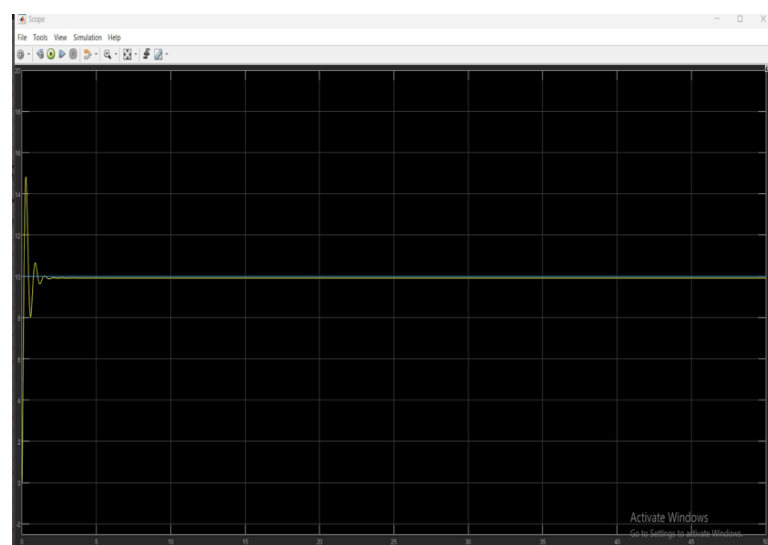


Figure I.5: System response after PID tuning using the Ziegler-Nichols method

Section II: Control Strategy

This chapter focuses on how the robot's movement is controlled through the independent actuation of its left and right wheels. The control strategy aims to adjust the rotational speeds of the wheels in such a way that the mobile robot follows a desired trajectory. To achieve this, we first derive the robot's linear and angular velocities based on the wheel speeds, and then formulate the kinematic model that governs its motion in the 2D plane.

II.1 Linear Velocity of the Robot:

$$v = \frac{r}{2} \times (\omega_r + \omega_l)$$

II.2 Angular Velocity of the Robot:

$$\omega = \frac{r}{L} (\omega_r - \omega_l)$$

Where:

v is the linear velocity of the robot [m/s]
ω_r is the angular velocity of the right wheel [rad/s]
ω_l is the angular velocity of the left wheel [rad/s]
r is the radius of the wheels [m]
L is the distance between the centers of the two driving wheels [m]

II.3 Motion Model in the Plane:

The robot's position in the plane is defined by coordinates (x, y) and an orientation angle θ . The rate of change of these quantities over time is governed by the following equations:

$$\begin{aligned} \dot{x} &= v \times \cos(\theta) = \frac{r}{2} \times (\omega_r + \omega_l) \times \cos(\theta) \\ \dot{y} &= v \times \sin(\theta) = \frac{r}{2} \times (\omega_r + \omega_l) \times \sin(\theta) \\ \dot{\theta} &= \omega = \frac{r}{L} \times (\omega_r - \omega_l) \end{aligned} \quad \xrightarrow{\text{Laplace Transform}} \quad \begin{cases} x = \frac{V(P) \times \cos(\theta)}{P} \\ y = \frac{V(P) \times \sin(\theta)}{P} \\ \theta = \frac{W(P)}{P} \end{cases}$$

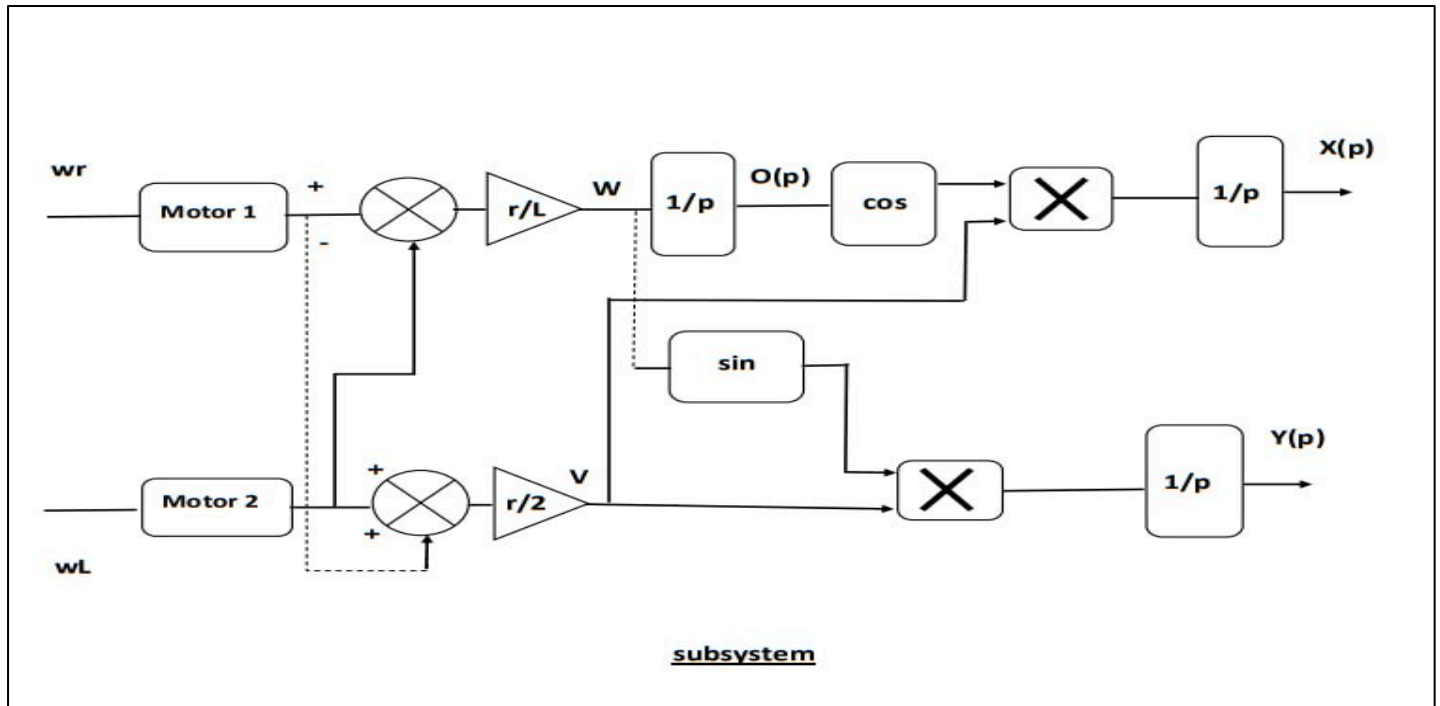


Figure II.2: Full vehical model

II.4 Simple Command Strategy:

In open-loop control, the robot's motion is determined by directly setting the left and right wheel angular velocities, producing different trajectories depending on their values.

- If $\omega_r = \omega_l \rightarrow$ the robot moves in a straight line forward, since both wheels rotate at the same speed and no rotation is induced.
- If $\omega_r > \omega_l \rightarrow$ the robot turns to the left. The right wheel spins faster than the left, causing the robot to pivot left around a point closer to the slower wheel.
- If $\omega_l > \omega_r \rightarrow$ the robot turns to the right. The left wheel spins faster, and the robot turns right around a point near the right wheel.
- If $\omega_r = -\omega_l \rightarrow$ the robot rotates in place (pure rotation), one wheel moves forward and the other backward.

II.5 Advanced Control Strategy: Following a Given Trajectory:

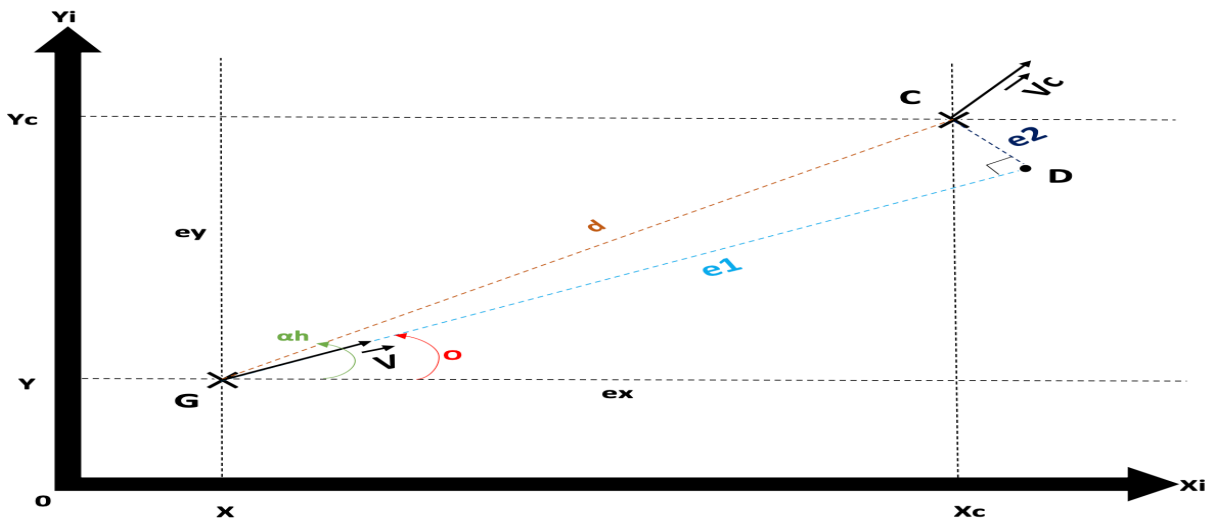


Figure II.3: Illustration of e_1 and e_2 Errors in a Robot Trajectory following control strategy

The Advanced Control Strategy aims to guide the robot along a predefined trajectory by using information about its current position and orientation. Unlike simple control, this method considers the error between the robot's current position (point G) and the desired target position (point C). Two main errors are defined: e_1 , the longitudinal error, which represents the remaining distance along the trajectory, and e_2 , the lateral error, which measures the perpendicular deviation from the path. The goal is to minimize these errors, ideally achieving $e_1 = 0$ and $e_2 = 0$, so that the robot reaches the target while staying aligned with the desired path. Based on this approach, e_1 is directly related to the linear velocity of the robot, since it measures how far the robot still needs to move forward, while e_2 influences the angular velocity, as it reflects how much the robot must rotate to stay on the trajectory. This strategy relies on a more precise motion model and may include error-based control laws to continuously correct the robot's movement.

II.6 Control and error computation equations:

$$e_1 = d \times \cos(\alpha h - \theta) = 0 \Rightarrow d = 0$$

$$e_2 = d \times \sin(\alpha h - \theta) = 0 \Rightarrow \theta = \alpha h$$

$$e_x = x_c - x$$

$$e_y = y_c - y$$

$$d = \sqrt{e_x^2 + e_y^2}$$

$$\tan(\alpha h) = \frac{e_y}{e_x} \Rightarrow \alpha h = \arctan = \frac{e_y}{e_x}$$

$$w_r = w \times \frac{L}{2r} + \frac{v}{r}$$

$$w_l = \frac{v}{r} - \frac{w \times L}{2r}$$

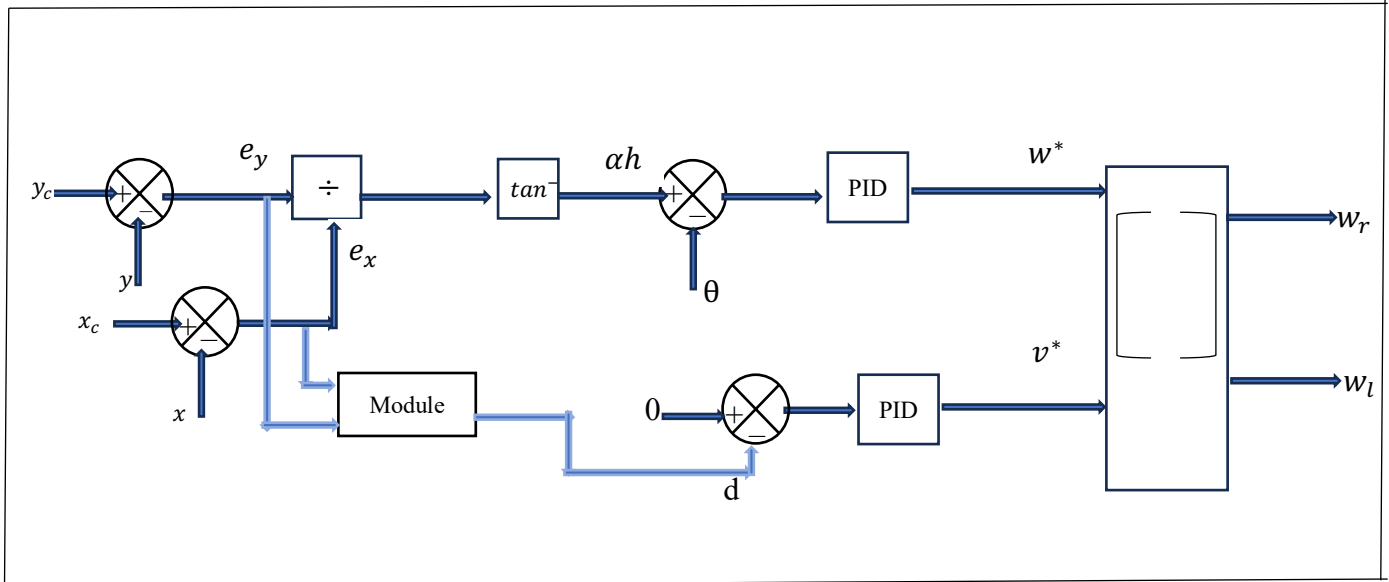


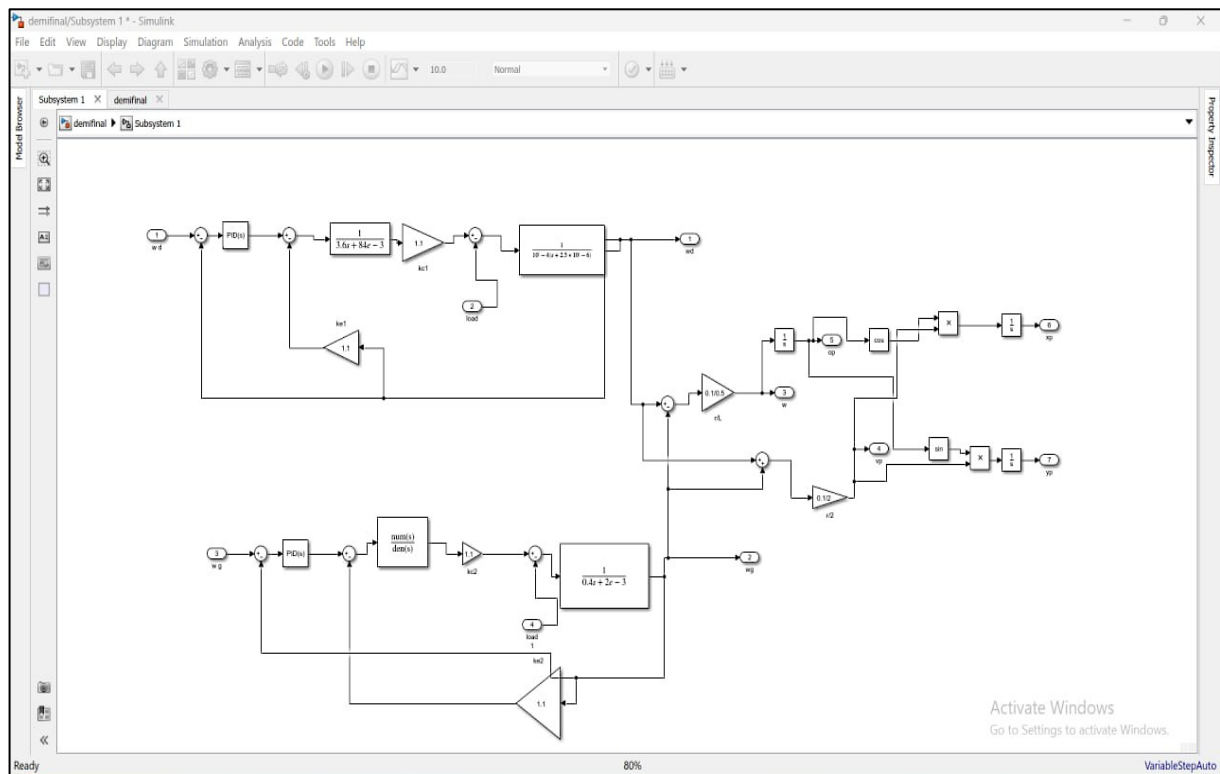
Figure II.4: Converting trajectory into speed input

Section III: Simulation and results

In this chapter, we present the simulation model developed using Matlab/Simulink, which aims to validate the dynamic behavior of the mobile robot and evaluate the performance of the proposed control strategy. The main objective is to implement the control laws discussed in the previous chapters and observe the system's response in various trajectory tracking scenarios.

III.1 Simulink Model:

The simulation is organized into two main subsystems: the complete control model and the vehicle model representing the robot's kinematics and dynamics.



FigureIII.1: Vehicle Model (Subsystem)

This subsystem describes the robot's kinematic behavior based on the mechanical equations presented in Chapter 1. It translates the wheel angular velocities into the robot's linear and angular motion (x , y , θ), allowing us to observe the real-time position and orientation of the mobile platform.

To validate the behavior of the vehicle subsystem, two basic movement scenarios were tested. The corresponding results are shown below:

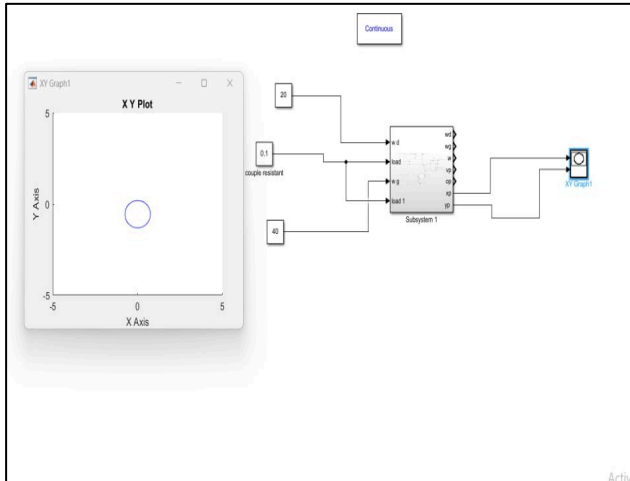


Figure III.2: trajectory – circular motion ($\omega_r \neq \omega_l$)

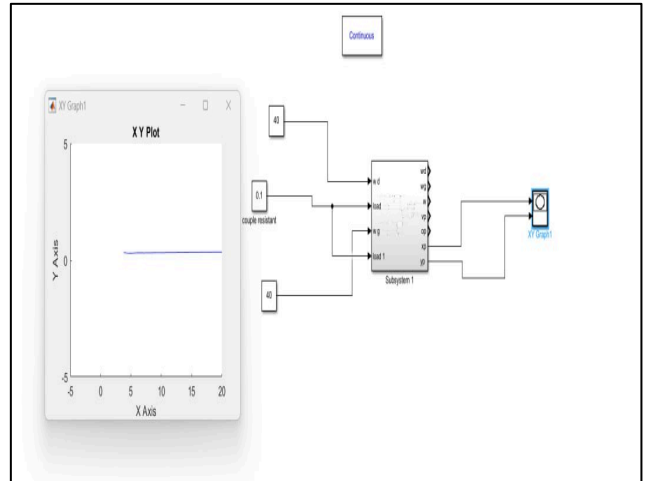


Figure III.3: XY trajectory – straight motion ($\omega_r = \omega_l$)

These results confirm that the subsystem model correctly simulates the basic kinematic behavior of the differential-drive robot.

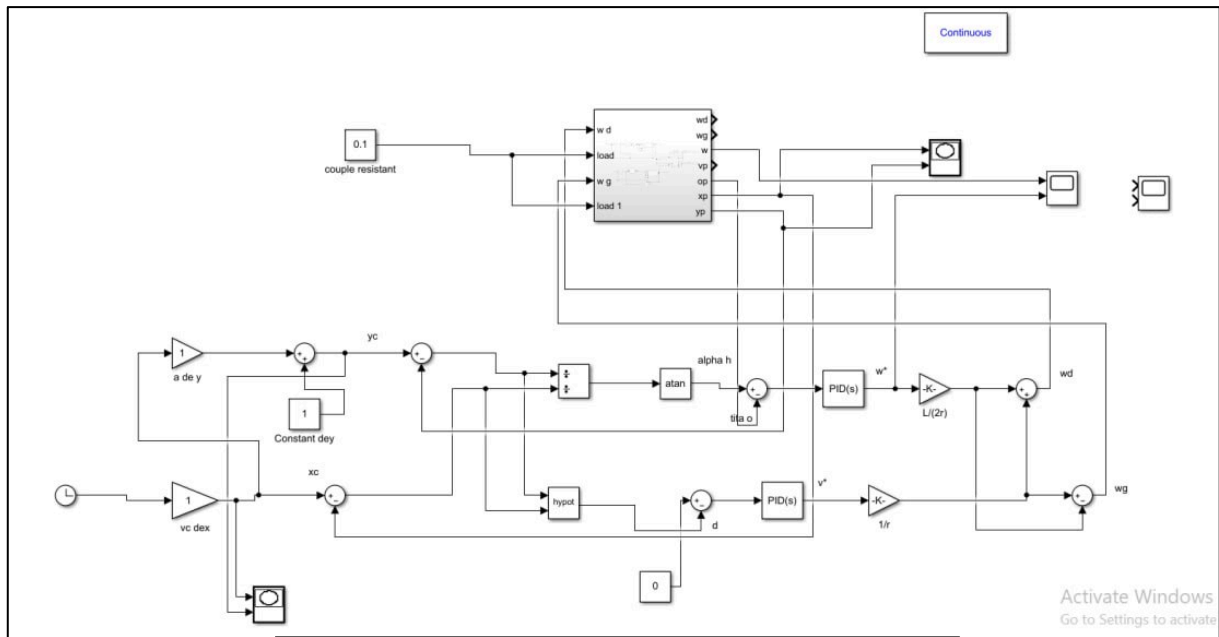


Figure III.4: Complete Control Model

This figure illustrates the structure of our advanced control strategy, which calculates the angular speed (ω) and linear speed (v) of the robot. From these values, the corresponding left and right wheel angular velocities (ω_l and ω_r) are determined. These wheel speeds are then used to control the motion and orientation of the vehicle along the desired trajectory.

III.2 Reference Trajectory Tracking :

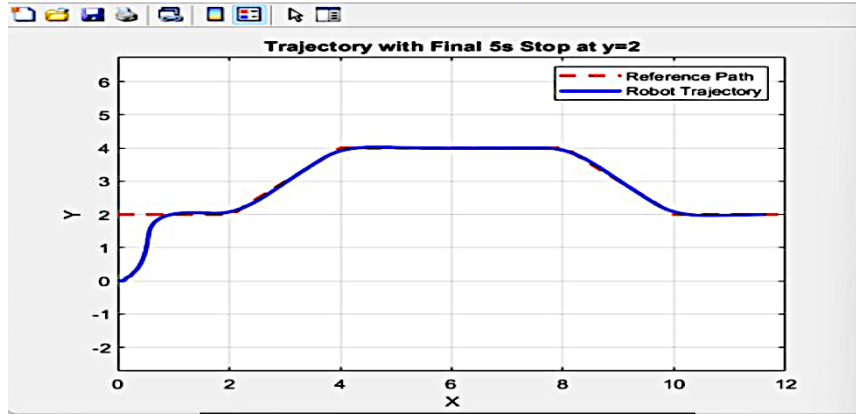


Figure III.5: Trajectory Tracking:
Robot vs Reference Path

This plot shows the robot's actual path (blue line) compared to the desired reference path (red dashed line). We observe that the robot starts following the trajectory immediately, but the tracking improves progressively. This behavior is due to the PID controller requiring a short time to stabilize and minimize the error between the robot's current position and the target path. Once the controller settles, the robot accurately follows the trajectory, even during curves or when temporarily stopping.

III.3 Wheel Angular Speeds Evolution:

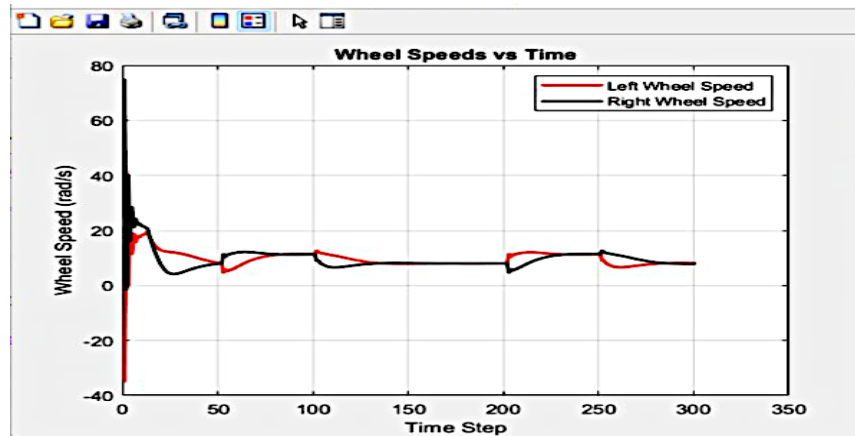


Figure III.6: Wheel Speeds vs
Time

This plot illustrates the evolution of the left (red) and right (black) wheel angular velocities over time. At the beginning, there is a sharp spike in wheel speed, which corresponds to a large initial error between the robot's starting position and the desired path. The controller responds by rapidly increasing the wheel speeds. As the error decreases, the speeds become more stable. When both speeds are equal, the robot moves straight. When they differ, the robot turns to adjust its orientation and stay aligned with the reference trajectory.

III.4 Linear Velocity Evolution :

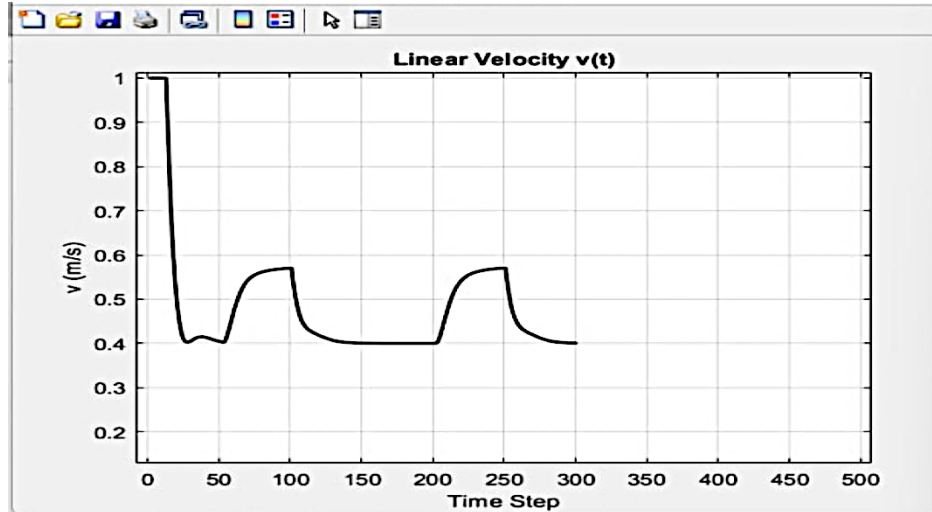


Figure III.7: Linear Velocity $v(t)$

This graph shows the evolution of the robot's linear velocity over time, calculated as $v = (r/2) \times (\omega_r + \omega_l)$. As we can see, the velocity follows the variations of the angular speeds of the wheels. When ω_r and ω_l increase together, the linear speed increases; when they decrease or remain constant, the linear speed stabilizes or drops. These changes reflect transitions in the robot's movement along the path.

III.5 Angular Velocity Evolution:

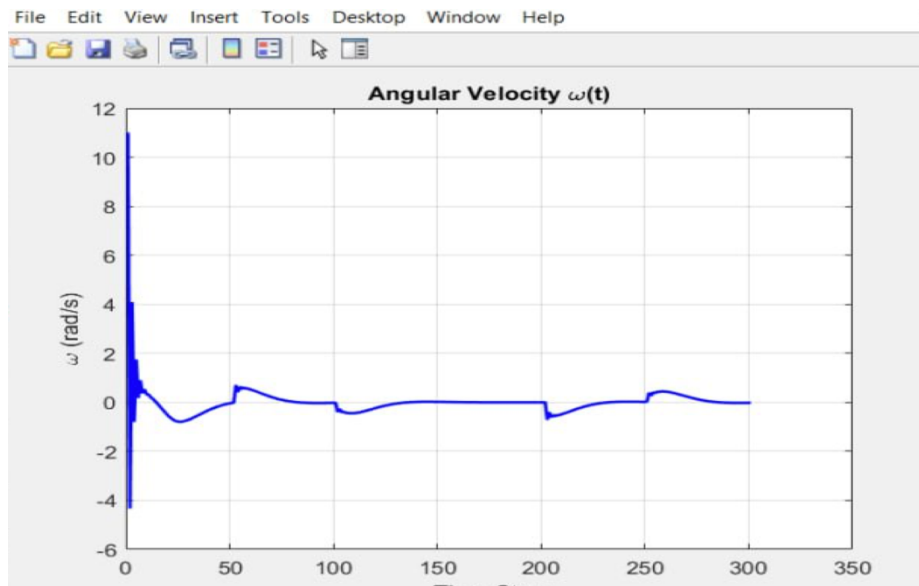


Figure III.8: Angular Velocity $\omega(t)$

This plot illustrates the angular velocity of the robot, which corresponds to the rate of change of its orientation (θ). Each time the robot changes direction, $\omega(t)$ deviates from zero. The sharp peak at the beginning indicates a large initial orientation error, which the controller attempts to correct quickly. When the robot moves straight, the angular velocity remains close to zero, confirming that there is no turning.

III.6 Conclusion of the Simulation Results:

Through these four figures, we observe that the implemented control strategy allows the robot to effectively follow the reference trajectory. The robot continuously adjusts its linear and angular velocities based on the difference between its current position and the desired path. The variations in the wheel speeds reflect the controller's effort to minimize position and orientation errors. The initial peaks in the curves are due to large initial errors, followed by stabilization as the robot aligns with the trajectory. These results confirm the effectiveness of the developed control system.

Conclusion

General Conclusion:

This final-year project focused on the modeling, simulation, and control of a differential-drive mobile robot powered by two independent electric motors. The work began with the development of the mechanical and electrical equations governing the behavior of the system, which laid the foundation for the design of an effective motion control strategy. Through MATLAB/Simulink simulation, we implemented and analyzed two types of control approaches: a basic open-loop control based on predefined wheel speeds, and an advanced closed-loop control designed to follow a desired trajectory by minimizing position errors in real time.

The results demonstrated the efficiency of the proposed control method and confirmed the robot's ability to accurately follow a reference path. The simulation validated that the system behaves as expected under both simple and advanced command strategies.

This project represents an important step toward the development of intelligent mobile robotic systems capable of precise navigation. Future improvements could include implementing the control on a physical prototype, integrating sensor feedback, or exploring more advanced control algorithms for dynamic and uncertain environments.